

Terminating Installed Communication State When the Control Path Dies: Deadline Expiry and Delivery-Bounded Configuration Leases for Intermittent Pre-Disaster Monitoring

Pre-disaster monitoring & emergency-communication working draft
agentic-communication repository, method v1.2 — September 19, 2026

Abstract

When a mountain-site backhaul fails, the network does not freeze: communication state that was installed *before* the failure keeps executing. Unacknowledged sensor records continue to be retransmitted and fill the few recovery access batches, and a dense sampling configuration that can no longer be countermanded keeps draining a battery through the night. We study where, and on what evidence, these two forms of *persistent installed state* can be terminated after the control path dies, for a battery-plus-solar LoRaWAN Class A deployment with an autonomous gateway, a cellular primary and a rate-limited BeiDou short-message backup. We formulate one decision problem—local termination under partial evidence—in which a node, holding only its clock, battery, cache, recent access receipts, and bounds supplied at install time, must either retain or release a record and either hold or relax a configuration, subject to no-false-release, energy survival, locality, and no-false-completion constraints. For records the termination time is the obligation’s business deadline, which the node knows unconditionally; we show the resulting source-local rule is bit-identical to standard Bundle-Protocol per-record expiry (BPv7 lifetime), yet its *cross-segment placement* decides the gain: source-local expiry raises outage on-time delivery 202→434 and 2.52-fold over ten seeds, while suppressing expired sends at the gateway alone back-pressures access (−8). For configurations the termination time is *not* locally known—it depends on mission revisions that travel on the failed path and on return-slot geometry held at the gateway. A fixed clock night-guard suffices when the outage aligns with sunset and energy is ample, but under overcast skies it leaves 20–31 nodes to discharge fatally; a fixed config TTL that is short enough to save those nodes falsely releases 183 still-deliverable obligations in a different outage phase. A *delivery-bounded lease*, whose installed bound is the last time the current profile’s data can still meet an obligation (deadline aligned to a return slot), removes the deaths in every phase without false release. These mechanisms are ordinary primitives with a re-derived bound; an LLM controller is used only formatively and reveals the interface fault—LoRa access receipts read as backhaul state—that the design targets. All claims are from deterministic, repository-reproducible runs; end-to-end and cross-model agent validation is specified as future work.

1 Introduction

A concrete failure, not an abstraction. Consider a field node running an authorised dense sampling profile. While the cellular backhaul is up the centre sends the configuration and the node applies it at its next LoRaWAN Class A receive window. The backhaul then fails for sixteen hours.

Three things keep happening with no control path to stop them. First, the node keeps sampling and transmitting at the dense rate, drawing energy the battery may not have through the night; a downgrade authored after the failure cannot reach it. Second, records already cached at the node are, under the standard hold-until-acknowledged edge discipline, retransmitted oldest-first on every narrow access opportunity, including records whose business deadline has already passed. Third, the centre and gateway cannot tell whether the missing data were never sampled, are sitting at a node, have reached the gateway, or simply lack a return slot, and they cannot tell a command that is queued, in transit, or applied. The network is not stuck; *installed state is still executing*, and its execution spends energy, access slots, and metered short-message bytes on work that can no longer meet any obligation.

This is a communication-systems failure with a specific shape: **the termination of a persistent installed state depends on a control/acknowledgement path that has itself failed**. SateRIoT observes that lossy satellite windows waste COTS-radio retransmissions and redesigns link estimation and multi-hop sharing around that measured behaviour [1]; semantic HARQ reuses the mature HARQ primitive but changes its error criterion to match a new communication object [2]. In the same spirit we do not seek a new primitive for its own sake. We ask which ordinary mechanism already terminates each kind of installed state, and where a genuine gap remains after the ordinary mechanisms are applied fairly.

Two kinds of persistent installed state. A **record state** is an unacknowledged sample in the node cache, occupying a slot in every future access batch until it is acknowledged or discarded. A **configuration state** is the node’s current sampling/reporting profile, installed by an applied command, which continuously drives energy and airtime until a newer command or a local condition ends it. Both are created while connected and both outlive the connection. They differ in what evidence terminates them: a record’s useful lifetime ends at an absolute business deadline the source can compute from the public mission cadence, whereas a configuration’s useful lifetime ends when the data it produces can no longer be delivered—a fact that depends on mission *revisions* (which reach the field over the failed backhaul) and on the geometry of future return opportunities (which the gateway, not the node, holds). This asymmetry, made precise in §4, is the paper’s technical centre.

What strong ordinary mechanisms already settle. Before proposing anything we assemble deliberately strong baselines with the same legal information: energy-aware feed-forward sampling, an omniscient offline packer, rolling-horizon MPC, max-coverage/recency backup packing, earliest-deadline-first and obligation-greedy queues, piecewise non-prescient mission views, and symmetric resource relaxations. Over the policy families we tested, re-optimising collection, scheduling, or packing under fixed resources yields no robust on-time-delivery gain; the large fixed-resource gains require adding backhaul capacity or energy. We report this as a scoped negative result, not as an optimality theorem. Two ordinary termination mechanisms also work well. (i) *Per-record expiry*. Releasing a cached record when its business deadline passes is standard Bundle-Protocol lifetime management [3]; we prove a prior “certificate-driven purge” is bit-identical to it. (ii) *A local clock/energy guard*. A node-local rule that relaxes dense sampling at dusk removes overnight deaths whenever the outage aligns with sunset and the battery is adequate. We explicitly credit these as mature primitives.

The residual that is not settled. Two controlled experiments isolate what the ordinary mechanisms do *not* cover. (1) Releasing expired records at the *gateway* (stop sending them, leave the

source cache untouched) withholds the acknowledgement, so the expired record keeps filling the node’s access batch; on-time delivery moves *down* 202→194. The disposition must be consistent across the access and backhaul segments, which single-hop AoI discard and hop-local bundle expiry do not address. (2) The clock guard acts only at dusk. When an authorised daytime downgrade falls inside an outage, the node runs the dense profile for hours after its data have ceased to be deliverable; under overcast skies this lag is fatal (20–31 nodes), yet a naive short config TTL that relaxes early enough to save those nodes falsely releases obligations still deliverable over the backup in a differently phased outage (183 obligations lost). No fixed TTL value is safe and sufficient across phases. The residual is therefore not “terminate state”—ordinary expiry and guards do that—but **computing the configuration’s termination bound from delivery geometry the node does not have, and installing it while connected so it can be enforced locally while disconnected.**

Contributions.

1. **A single termination problem for persistent installed state (§3,§4).** We model records and configurations as state installed while connected and executed while disconnected, define each state’s release time from the obligation chain, and state one local decision problem under four constraints (no-false-release, energy survival, evidence locality, no-false-completion), with a three-class confirmed/unconfirmed/unreachable statement discipline that makes *unknown* a first-class outcome.
2. **Record termination is standard expiry; cross-segment placement is the gain (§5.1,§7).** We prove deadline-aligned source expiry is bit-identical to BPv7 per-record lifetime on ten seeds and isolate the system contribution—an end-to-end business-deadline basis and source/gateway consistency—which doubles outage on-time delivery and drives expired backup records to 1, while gateway-only suppression moves cost to the access segment. A full-horizon time-aware attribution corrects an earlier accounting that had labelled 830 late-access failures as backhaul-capacity failures.
3. **A delivery-bounded configuration lease for the residual (§5.2,§7).** We reuse the ordinary config-TTL/lease primitive but replace its fixed-duration bound with the last deliverable time of the current profile’s data (obligation deadline aligned to a primary/backup return slot), computed at the gateway while connected and enforced at the node with no downlink. Across outage phases and harvest levels it removes the 20–31 overcast-sky deaths the clock guard leaves and avoids the 183-obligation false release of the best fixed TTL, while in ample-energy regimes it only removes wasted dense work.
4. **A formative real-agent study that locates the interface fault (§7.6).** Symmetric accounting of 1089 live LLM decisions shows LoRa access receipts being read as backhaul state in both optimistic and pessimistic directions; an unknown-aware projector removes the healthy-link false assertions in offline replay. The agent is a *controller candidate*, not part of the problem definition; end-to-end and cross-model runs are specified as future work.

We do not claim an LLM out-optimises a solver, a new discard algorithm, or a global scheduling upper bound. The deterministic mechanisms are in-layer tools available equally to rules, solvers, and agents.

2 Background, scenario, and standards grounding

Deployment. Fourteen sensor nodes in two groups monitor displacement (and rainfall) on a slope. Nodes sample and upload to a local gateway over LoRaWAN Class A; the gateway forwards to a central server over cellular backhaul, with a BeiDou short-message service as a sparse, metered backup that carries field-to-centre messages only (it is not a downlink configuration channel). Nodes are battery plus small solar. The mission is pre-disaster and routine, with authorised warning-level changes; no component here judges landslide risk. Node count, role mapping, and disturbance parameters follow the frozen Task v1.1 instance manifest; an earlier 16-node sketch is not used as a field fact.

Standards-grounded authority. We use two Chinese geological-hazard monitoring industry standards, DZ/T 0460–2023 and DZ/T 0450–2023, retrieved and clause-audited in the repository [4, 5]. Relevant clauses: §5.3.3 permits remote dynamic adjustment of sampling and upload frequency by warning level (no numeric levels are given, so the periods we use are declared research choices); §8.4.1 defines four levels (red/orange/yellow/blue) and §8.4.2 permits upgrade, downgrade, or termination after a conference; §5.3.7 calls for dual-mode communication under weak coverage in high-risk extreme weather—a conditional clause that appears once, not a blanket dual-channel mandate, and the communication-layer standard does not impose it. The BeiDou open-service specification (BDS-OS-PS-3.0) [6] reports $\geq 95\%$ success, average latency under 2 s, an average service frequency of one message per 30 s, and a single-message ceiling of 14,000 bits, while noting that realised frequency and maximum length are constrained by registration parameters—the rate is a provisioned quantity. These clauses bound what the system may lawfully do: re-compile frequency within granted levels and request a conference change, never silently drop an obligation or declare risk.

Operating point. The backup uses one 78-byte packet every 1200 s (20 B header, 58 B payload; a displacement sample is 6 B and rainfall 4 B, so roughly 9–10 samples fit per packet). The elevated (yellow) task uses a 300 s period with a delivery deadline two periods after window open, i.e. a ≈ 600 s delivery window—shorter than the 1200 s backup slot spacing. The 200 B single-packet ceiling and the ≥ 1 min period in DZ/T 0450 (§7.4.3.2, §7.4.2.3, with a ≥ 5 s inter-message gap at §7.4.3.5) belong to different service-formulation sections and are not combined into a single “BeiDou operating point”; the 1200 s/78 B point is a declared research choice.

3 System model

Time is slotted at $\Delta=60$ s; $t=0$ is 06:00 local, so $t=12$ h is 18:00 (sunset) and the solar cycle is explicit. Sets of nodes $\mathcal{N}$, measurands $\mathcal{M}$, and a gateway g compose the field; a centre c lies beyond the backhaul.

Obligations and the delivery chain. A monitoring obligation is $o = (n, m, [l_o, h_o], d_o)$ over node n , measurand m , sampling window and absolute delivery deadline. A routine obligation at period P has $h_o - l_o = P$ and $d_o = h_o + P$ (one period of grace). A sample s fulfils o only if it matches (n, m) , is taken in window, and is received centrally by d_o . Each sample traverses

$$\underbrace{\text{qualified sample}}_{\text{node}} \rightarrow \underbrace{\text{heard at } g}_{\text{LoRa access}} \rightarrow \underbrace{\text{returned by } d_o}_{\text{cellular / short-message}} \rightarrow \underbrace{\text{central evidence}}_{\text{fulfilled}}.$$

For piecewise missions the first segment uses a closed interval and later segments half-open windows, so a boundary sample cannot double-cover adjacent windows after a level change.

Links, outages, and who can act. The LoRa access link (node→gateway, with Class A downlink windows) succeeds per opportunity with probability $p_a=0.74$; the primary backhaul (gateway→centre) is good with probability $p_b=0.62$ when not in a declared outage. A control outage is an interval $\mathcal{O} = [a, b)$ during which the primary is unavailable to centre→field traffic: centre commands and mission-table segments are refused at admission (never transmitted), and the short-message backup cannot carry downlink configuration. During $\mathcal{O}$ the only entities that can change field communication behaviour are the *node* (on its clock, own battery, cache, recent access receipts, and bounds carried by already-applied commands) and the *gateway* (on samples it has actually heard, its forwarding log, and backup slots used). The centre retains authority to authorise changes but no channel to install them; a revision authored in $\mathcal{O}$ reaches the field only at b .

Persistent installed state. Two state types are installed while connected and executed while disconnected.

- *Record state.* Each unacknowledged sample s at node n has creation time t_s , a business deadline $d(s) = (\lfloor t_s/P \rfloor + 2)P$ under the locally known cadence, and occupies one of $K=240$ cache slots and up to $B=32$ positions in an access batch. Under the modelled edge discipline it leaves the cache only on central acknowledgement (modelled synchronously without airtime, equally favouring every queue rule); until then it is re-sent oldest-first.
- *Configuration state.* The node runs a profile $q(t) \in \{P_0, P_1\}$ (sparse $P_0=600$ s, dense $P_1=300$ s) for both sampling and reporting, installed by an applied command that carries a version. A command is *applied* only when a later node report confirms the new profile (Class A latency); a queued or in-transit command changes nothing. While applied, $q=P_1$ draws additional sampling and uplink energy every tick until something terminates it.

Energy. A sample costs $e_s=4.7 \times 10^{-4}$ Wh and an uplink $e_u=2.33 \times 10^{-5}$ Wh; battery capacity is $C=0.05$ Wh. Solar input is a 12 h half-sine with peak η (nominally 0.03 Wh/h) plus per-node/hour stochastic cloud occlusion. A node whose state of charge reaches zero *permanently* dies in the main configuration (an absorbing state). Holding dense sampling over a 12 h night draws ≈ 0.071 Wh $> C$.

Release time of installed state. For each state x define its release time $t_{\text{rel}}(x)$ as the earliest time after which keeping x alive can no longer contribute to a fulfilled obligation.

- For a record, $t_{\text{rel}}(s) = d(s)$: past the business deadline the record cannot fulfil any obligation, so further retention or retransmission has zero delivery value.
- For a configuration profile, $t_{\text{rel}}(q)$ is the last deadline of an obligation the profile's data can still meet through *some* feasible path:

$$t_{\text{rel}}(q) = \max_{o: \text{profile } q \text{ produces } o} \min\{d(o), \tau_{\text{last slot}}(o)\}, \quad (1)$$

where $\tau_{\text{last slot}}(o)$ is the latest primary or backup return opportunity at which a qualified sample for o can be heard and forwarded by $d(o)$. Unlike $d(s)$, this bound combines mission timing (which obligations the current profile serves) with return-slot geometry (when they can be carried), neither of which is fully observable at the node during $\mathcal{O}$.

Service loss. The objective counts deadline-valid obligation delivery (service), permanent node deaths, and the resource cost of state that outlives its release time—wasted samples, access attempts, short- message bytes, and energy. Offline, each unfulfilled obligation is charged to its *first* failing stage under an optimistic model that grants every later assumption, giving four mutually exclusive segments: S_{energy} (no qualified sample), S_{access} (sampled but not heard at the gateway by the deadline), S_{time} (heard but no return opportunity exists), S_{cap} (an opportunity exists but capacity/congestion cannot carry it). An online observer that cannot see a stage must report *unknown*, never guess it.

Three statement classes. Every feasibility claim is one of: **confirmed** (an independent report shows the target config, or a matching sample is received centrally); **unconfirmed** (a command is in flight or receipts are incomplete—this implies neither that it took effect nor that the channel is down); **unreachable** (the current path is known down from explicit network-side state). A LoRa access receipt (“a node was heard”) is evidence about the access link only and can never by itself establish backhaul reachability.

4 Problem formulation

During an outage $\mathcal{O}$, at each tick t each acting entity chooses, for the state it can touch, an action: for a record $a_t(s) \in \{\text{RETAIN}, \text{RELEASE}\}$; for the configuration $a_t(q) \in \{\text{HOLD}, \text{REVERT}\}$ (revert dense to the last confirmed sparse profile). Let $\mathcal{E}_e(t)$ be the evidence entity e lawfully holds at t (for a node: clock, own SoC, cache contents, recent Class A receipts, and any bound carried by commands applied before a ; explicitly *not* future mission-table segments, simulator-wide samples, or backhaul truth). The local termination problem is

$$\textbf{Problem 1.} \quad \min_{\{a_t\}} \sum_o \mathbf{1}[o \text{ unfulfilled}] + \lambda_d \sum_n \mathbf{1}[n \text{ dies}] + \lambda_w \sum_{x: t \geq t_{\text{rel}}(x)} \text{cost}(x \text{ kept}) \quad (2)$$

$$\text{s.t. (no-false-release)} \quad a_t(x) = \text{RELEASE/REVERT} \Rightarrow t \geq t_{\text{rel}}(x), \quad (3)$$

$$\text{(energy-survival)} \quad \text{SoC}_n(t) > 0 \quad \forall n, t, \quad (4)$$

$$\text{(locality)} \quad a_t \text{ is a function only of } \mathcal{E}_e(t), \quad (5)$$

$$\text{(no-false-completion)} \quad \text{unknown feasibility} \Rightarrow \text{report unconfirmed or unfulfillable, never fulfilled or unreachable.} \quad (6)$$

The tension is between (3) and (4) under the locality restriction (5). Releasing too early violates (3) (work that could still be delivered is abandoned); releasing too late violates (4) (the node keeps paying for dense data after they have become undeliverable, and discharges fatally). The whole difficulty is that the two state types observe t_{rel} very differently.

Record sub-problem: t_{rel} is locally certain. Because $d(s)$ uses only the public cadence, the node evaluates $t_{\text{rel}}(s) = d(s)$ with certainty and no downlink. Constraint (3) is the exact test $d(s) \leq t$, and RELEASE frees both the backup send and the access batch. The optimal local rule is ordinary per-record expiry; §5.1 shows a certificate-keyed variant is the same rule and that the only design choice with measurable consequence is *where the release is executed*, via constraint (5) and the cross-segment acknowledgement coupling.

Configuration sub-problem: t_{rel} is not locally observable. During $\mathcal{O}$ the node cannot compute $t_{\text{rel}}(q)$ from (1): mission revisions that change which obligations q serves travel on the failed backhaul, and return-slot geometry sits at the gateway. Three local approximations are available, each flawed:

- *Clock guard*: revert at sunset. Correct when $\mathcal{O}$ aligns with dusk and dense data are undeliverable by then; blind to a daytime downgrade and to overcast days with little harvest (§7.3).
- *Fixed config TTL*: revert Δ after install. A single Δ cannot track (1): short Δ violates (3) when the dense window is long; long Δ violates (4) under tight energy when the window is short (§7.3).
- *Local energy guard*: revert on a SoC threshold computed at the node. Necessary and deployed as a safety net, but reactive—it acts only once energy is already low and cannot tell whether the data being sacrificed were still deliverable.

Delivery-bounded lease (our mechanism). While connected, the gateway computes (1) for the profile it is about to install, using obligations it knows and the primary/backup slot schedule, and attaches the resulting absolute bound τ_L to the configuration command. The node stores τ_L at apply time and reverts locally at $t \geq \tau_L$ with no downlink. The bound uses only information valid at install time and never forecasts a future authorisation; if a later connected revision changes the profile, a fresh command carries a fresh lease. This is the ordinary config-lease/TTL primitive with its bound re-derived from delivery geometry, exactly as semantic HARQ reuses HARQ but changes its criterion [2]: it is the minimal local object that makes (3)–(4) jointly satisfiable without assuming a surviving control path. When even the lease cannot keep the node alive, the honest outcome under (6) is to revert to the safe sparse default and mark the interval’s obligations unfulfillable—not to report them complete.

Why this is a communication problem, not an agent problem. Problem 1 is defined over network state, evidence locality, and return geometry; it has an optimal oracle solution and ordinary rule-based solutions independent of any LLM. An agent is one possible *solver* of the same decision (and a convenient probe of where the interface misleads a planner); nothing in the objective or constraints references a model. We therefore evaluate the mechanisms deterministically and treat the LLM study as formative.

5 Mechanisms

5.1 Record termination: deadline-aligned expiry

At each upload opportunity the node deletes cached records with $d(s) \leq t$ and sends survivors oldest-first. We implement two formulations: **deadline-purge**, keyed to the obligation cadence, and **generic-expiry**, an ordinary BPv7-style per-record lifetime assigned at creation, expires = $t_s + ((P - t_s \bmod P) + P) = d(s)$, referencing no obligation id, slot geometry, or certificate. A gateway variant locally acknowledges records it will not send. EDF and obligation-greedy queues only reorder and never release; AoI **latest-only** releases indiscriminately (keeps only the newest). Dropping stale/expired traffic is well established in AoI packet management and DTN bundle expiry [7, 8, 9, 3]; the sensor-transport practice of suppressing traffic whose delay bound is exceeded is likewise prior art [10]. We claim no new discard algorithm. The system-level content is the end-to-end business-deadline basis (not packet age, hop TTL, or an estimated single-packet delay), zero-downlink source placement, and—decisively—cross-segment consistency, because the acknowledgement coupling means a gateway-only release does not free the source cache.

5.2 Configuration termination: local guard and delivery-bounded lease

Local clock/energy guard (mature). On every tick the node, on its local clock, clamps sampling/report to sparse at night ($\text{hod} < 06:00$ or $\geq 18:00$) and leaves daytime commands untouched; an optional computed energy ledger (remaining dark hours $\times$ the dense-vs-sparse extra draw versus the node’s own SoC) additionally reverts when dense operation would not survive the night. This is a standard time/energy local rule and models the gateway’s “pre-provisioned autonomous rules”; it is off by default and bit-identical when off. **Delivery-bounded lease (the residual mechanism).** The gateway computes τ_L (Eq. 1) while connected: for the current dense profile it takes the last obligation the profile produces before the next known revision, sets that obligation’s qualified-sample and return-slot deadline, aligns to the latest feasible primary/backup slot, and attaches τ_L to the install command. The node enforces revert at τ_L alongside the clock guard; the guard remains the always-on safety net for conditions the install-time bound cannot see (e.g. an unforeseen weather collapse). A pre-provisioned full schedule is the degenerate case in which τ_L for every segment is known at $t=0$ and a local schedule executor suffices with zero centre traffic.

5.3 Partial-observability projector and statement discipline

At compile time the centre holds the obligation table, the *current* (never forecast) network-side outage state, and public parameters; at mission-table arrival the gateway holds only what it has heard. A structural no-return certificate (S_{time}) is emitted from slot geometry alone and is explicitly conditional on the primary being unavailable; geometry never infers an outage. An arrival-time projector labels a missed obligation only when local evidence separates the stages and otherwise returns unknown. An unknown-aware centre projector emits *unreachable* solely from an explicit network-side field (which the current harness does not yet provide, so it emits *unconfirmed*), and replaces any blanket night kill with the computed energy ledger so that genuine energy-infeasibility is distinguished from conservative advice.

6 Prototype and methodology

All mechanisms run in one Python event-stepped simulator (60 s ticks) implementing LoRaWAN Class A downlink windows, versioned two-field configuration commands (sampling interval and reporting period as independently delivered fields, with cross-generation mixed configs detected), the hold-until-acknowledged edge cache, max-coverage backup packing, stochastic solar/cloud, absorbing brownout, and the non-prescient piecewise mission-view gate. Disabling the backup reproduces the v1.1 simulator bit-for-bit (five anchored invariants); the local guard and lease are default-off. **Two controlled episodes** are used. *Episode I* (the v1.1 manifest point): upgrade to yellow at $t=6$ h, primary outage 4–20 h, the mission table reaching the gateway at 20 h; it drives the frontier, attribution, record-expiry, placement, and live-agent results. *Episode II* (a deterministic configuration-termination scenario, identical parameters except schedule, outage phasing, and swept harvest): to place a daytime downgrade *inside* an outage it uses yellow at $t=2$ h with a conferred downgrade at $t=6$ h and outage 4–20 h (phase A), or yellow at $t=1$ h, downgrade at $t=8$ h, outage 6–22 h (phase B); $t=12$ h is sunset. Episode II is an explicitly constructed controlled comparison for Problem 1’s configuration sub-problem, not a new field claim. We report node deaths (deterministic/highly robust) as totals over seeds 0–2 and treat sub-point mean-service differences at $n=3$ as not significant; seeds 6–9 are a reused held-out confirmation set, not an untouched test set. Every figure is regenerated by a registered repository script and the suite reports 18/18 self-checks.

7 Evaluation

7.1 Where the fixed-resource loss actually is

Over Episode I’s 7560 routine obligations the strong maxcov/FIFO baseline delivers 3025 (0.400). Symmetric per-obligation counterfactuals widen one physical resource at a time (never the policy) and report set differences over identical obligation ids. Relaxing backhaul to unlimited gives 4740 (0.627), relaxing energy (0.50 Wh, always-dense) gives 4089 (0.541), and both gives 6023 (0.797); 1715 obligations are blocked by backhaul capacity alone, 1128 by battery alone, and 155 (2.0%) by both (Table 1). These are intervention results on the implemented strategies, not an exhaustive optimum; we make no zero-remainder or global-optimality claim. They establish that the large throughput gains are physical, which is precisely why we then target *state termination* rather than a smarter scheduler.

Run	Sampling/Energy	Backhaul	Delivered
base	dayfeed, 0.05 Wh	1200 s/78 B	3025 (0.400)
O_{bh}	dayfeed, 0.05 Wh	unlimited	4740 (0.627)
O_{en}	always-300, 0.50 Wh	1200 s/78 B	4089 (0.541)
O_{bo}	always-300, 0.50 Wh	unlimited	6023 (0.797)

Table 1: Counterfactual interventions over 7560 obligations (Episode I). Set decomposition over the tested configurations; not an upper bound over all legal policies.

Full-horizon, time-aware failure attribution. Charging each of the 4535 failures to its first failing segment with “heard” required *by the deadline* (not at any later time) gives S_{energy} 2002 (44.1%), S_{cap} 1115 (24.6%), S_{access} 830 (18.3%), S_{time} 588 (13.0%). An earlier heard-ever accounting had instead reported S_{cap} 1945 and S_{access} 0: all 830 moved obligations were in fact sampled but first heard at the gateway after their deadline—late access, not backhaul capacity. The delivered count (3025) is identical under both accountings; the correction changes *where the loss is charged*, and therefore which mechanism could address it.

7.2 Record expiry: standard primitive, decisive placement

During the Episode I outage 75 backup packets carry 750 records; under FIFO 219 records arrive centrally already expired versus 261 on-time (45.6%; 1270 of 2782 payload bytes). Gateway-only suppression removes those sends but withholds the acknowledgement, and on-time delivery falls 202→194: cost is moved to the access segment. Source-local deadline expiry reverses the sign, raising outage on-time delivery 202→434 and the deadline-valid fraction 0.54→1.00. Across ten seeds (Table 2), expiry raises mean service 0.373→0.415—a paired +4.21 points (95% CI +3.64 to +4.79, positive on all ten)—and total outage on-time delivery 1691→4265 (2.52×), expired backup records 2538→1, and deaths 12→0; it beats AoI latest-only by +5.73 points (CI +5.06 to +6.39), because latest-only buys outage freshness by abandoning post-recovery coverage. Crucially, **deadline-purge** and **generic-expiry** are bit-identical on every seed (service, outage counts, deaths, and the delivered set): the record mechanism *is* standard per-record expiration, and its measured value is the business-deadline basis plus source/gateway cross-segment consistency, not a new discard algorithm.

Edge disposition (maxcov fixed)	Full svc	Outage on-time	Expired backup	Deaths
FIFO (baseline)	0.373	1691	2538	12
latest-only (AoI)	0.358	4250	0	0
generic-expiry (standard)	0.415	4265	1	0
deadline-purge (node)	0.415	4265	1	0

Table 2: Ten-seed Episode I sweep with the strong maxcov chooser fixed; only the edge disposition changes. The standard expiration and the certificate purge are bit-identical. Gateway-only suppression (not shown; same FIFO cache) gives 194 outage on-time at seed 0 vs. 202 FIFO and 434 source expiry—the cross-segment cost.

7.3 Configuration leases: deaths under overcast skies, false release under short TTL

We now hold record expiry fixed (generic-expiry + maxcov) and vary only the configuration termination in Episode II. The node profile trace confirms the clock guard holds dense until sunset ($t=12$ h): 12 samples/h in $[2, 12)$ h and 6/h thereafter.

The delivery bound and a no-waste point. In phase A the last yellow obligation is released just before the noon downgrade; its deadline, aligned to a 1200 s backup slot, gives $\tau_L=6.33$ h. Sweeping the local revert time (Table 3), reverting at $\tau_L=6$ h removes 468 of 978 (48%) post-deadline daytime dense samples with *zero* loss of yellow delivery (356/672) and no deaths; reverting as early as 5 h falsely releases 9 yellow obligations, marking the no-false-release boundary. At nominal harvest ($\eta=0.03$) service, deaths (0), and final SoC are unchanged: in this regime the lease buys resource/airtime efficiency, not service.

Local revert bound	Deaths (3 seeds)	Mean svc	Mean final SoC	Yellow delivered
<i>Phase A, overcast $\eta=0.012$ (upgrade 2 h, downgrade 6 h, outage 4–20 h)</i>				
clock guard (sunset 12 h)	20	0.5961	0.066	849
fixed TTL 4 h (revert 6 h)	0	0.5970	0.122	849
fixed TTL 6 h (revert 8 h)	0	0.5970	0.122	849
fixed TTL 8 h (revert 10 h)	0	0.5971	0.117	849
delivery lease ($\tau_L=6.33$ h)	0	0.5970	0.122	849
<i>Phase B, overcast $\eta=0.012$ (upgrade 1 h, downgrade 8 h, outage 6–22 h)</i>				
clock guard (sunset 12 h)	31	0.5981	0.038	2020
fixed TTL 4 h (revert 5 h)	0	0.5881	0.122	1837 (–183)
fixed TTL 6 h (revert 7 h)	0	0.5994	0.122	1994 (–26)
fixed TTL 8 h (revert 9 h)	0	0.5996	0.122	2020
delivery lease ($\tau_L=8.33$ h)	0	0.5996	0.122	2020

Table 3: Configuration termination across two outage phases (seeds 0–2, generic-expiry + maxcov fixed). The clock guard leaves 20–31 overcast-sky deaths. No fixed TTL is safe in both phases: 4 h falsely releases 183 still-deliverable obligations in phase B, while 8 h is the phase B value and leaves phase A at minimal energy margin. The delivery bound τ_L tracks each phase’s last deliverable slot, reaching 0 deaths with no false release and the highest final SoC. At $\eta=0.03$ all arms have 0 deaths and the lease changes only wasted dense work; at $\eta=0.01$ even sparse operation is infeasible (all arms ≈ 40 deaths), which the system must report as unfulfillable rather than survive.

Tight energy turns the lag into service loss. Sweeping harvest (3 seeds): for $\eta = 0.03, 0.02$, and 0.015 , every bound gives 0 deaths and service ≈ 0.597 . At the overcast $\eta=0.012$ the clock guard’s sunset lag kills 20 nodes (service 0.5961, final SoC 0.066) while the lease at 6–8 h removes all 20 deaths (0.5970, SoC 0.122, near-doubled); reverting as late as 10 h also avoids deaths but at a smaller margin (SoC 0.117). At $\eta=0.01$ even sparse operation cannot be powered (≈ 40 deaths under every bound); the lease modestly raises pre-death service (0.530 $\rightarrow$ 0.577) but cannot save nodes—the correct output is an unfulfillable declaration, not a false completion.

No fixed TTL is safe across phases. Phase B needs the bound near 8.3 h (the long dense window), where a 4 h TTL relaxes at 5 h and falsely releases 183 yellow obligations (service -1.1 points) and a 6 h TTL still loses 26; phase A needs it near 6.3 h, where an 8 h TTL only just survives at the tightest energy (SoC 0.117). The same pattern holds at nominal harvest (the 4 h TTL still loses 183 obligations). The delivery bound attains each phase’s optimum automatically because it is computed from obligations and slots rather than guessed as a duration. This is the mechanism’s measured increment over a plain config TTL: phase/rate-adaptive termination under constraints (3)–(4), with a bound that is auditable (“stop because the data cease to be deliverable here”) rather than empirical. We report it as a point-specific, deterministic result over the tested phases; a wider sweep of backup rates and node counts is future work.

7.4 What the field can attribute when the table finally arrives

In Episode I the delayed mission table reaches the gateway at $t_a=20$ h, by which point 2136 yellow obligations are past deadline. Offline time-aware attribution labels all 2136 correctly (S_{time} 588, S_{cap} 327, S_{access} 650, S_{energy} 571). Using only evidence the gateway lawfully holds by t_a labels 1138/2136 (53.3%)—all geometric S_{time} (588), all in-time-not-returned S_{cap} (327), and 223 provably late S_{access} —with 100% precision on the labelled subset; 998 (46.7%) remain unknown, whose offline truth is 571 never-sampled and 427 heard only after t_a (Table 4). “Never sampled” and “sampled but not yet heard” are inseparable without evidence the gateway does not have, and must stay unknown (the original G5 constraint). Compile-time geometry, which needs no sample, emits 602 conditional no-return certificates of which 588 are true S_{time} (the 14 recovery-edge cases are excluded rather than certified); an optimistic-energy capacity certificate is impure (552 correct S_{cap} , 571 actually S_{energy} , 193 false positives), so capacity is left to the gateway. The design consequence is exactly Problem 1’s: local record/config action must not wait on resolving these unknowns.

Segment	Offline time-aware truth	Online gateway-local at t_a
S_{time} (no return slot; geometry)	588	588
S_{cap} (heard by deadline, not returned)	327	327
S_{access} (heard after deadline / late)	650	223
S_{energy} (no qualified sample)	571	— (unknown)
U (not separable online)	—	998
Total	2136	2136
Labelled coverage	100%	53.3% (1138/2136)
Precision on labelled subset	1.000	1.000

Table 4: Arrival-time attribution of the 2136 upgraded obligations already missed when their table reaches the gateway (Episode I). The online projector is exact but partial; 427 matching samples are first heard after t_a and 650 after their deadline.

7.5 Enforcement placement determines survival

A centre-side feasibility *envelope* (a pure filter over any planner using only confirmed configs, hearing counts, the authorised requirement, and the clock) removes the command storm but cannot save the tightest seed: a dusk downgrade must cross the backhaul and a Class A window it misses. The same “never stay dense overnight” rule is a late, refusable suggestion at the centre but an invariant enforced every tick at the node (Table 5). Naive centre compliance kills 13/14 nodes on all three seeds (39 total) at service 0.162; adding the node-local guard removes all deaths at unchanged service; with the guard present the envelope additionally cuts refused admission attempts to 222. A zero-command pre-provisioned rhythm reaches 0.360 service, matching every centre-compiled arm for these static missions—a scoped negative result on centre sampling control for pre-provisioned tasks, which is exactly the gap the delivery lease targets for *non*-pre-provisioned runtime changes.

Configuration (centre unless noted)	Mean svc	Deaths (3 seeds)	Cmds sent	Refused
comply (naive, no protection)	0.162	39	160	2488
dayfeed-c (centre day/night rule)	0.370	12	308	1256
envelope(comply), centre only	0.359	12	308	298
comply + local guard	0.366	0	1424	3364
dayfeed-c + local guard	0.368	0	258	1156
envelope(comply) + local guard	0.359	0	282	222
pure-local (pre-provisioned, 0 commands)	0.360	0	0	0

Table 5: Deterministic Episode I placement study (no LLM). The local guard removes all permanent deaths in the tested seeds at unchanged service; the centre envelope alone cannot. Refused commands are rejected at admission and never transmitted (command hygiene, not billing saving). Point-specific empirical results, not a survival invariant or optimal bound.

7.6 Formative real-agent study

We ran deepseek-flash live over the Episode I outage: three seeds each of an unaided agent (A0), the agent with an ordinary pending-command checklist (A0s), and a certificate agent (A1), plus two healthy-link controls, with a real model call at each 30-minute decision (99 per trace, 1089 decisions total); all arms share harness, legal observation, tools, history, and action space. This study motivates and stress-tests the interface, and is not used to establish the deterministic mechanisms.

Service and survival (noisy at $n=3$). A1 attains mean service 0.358 (level with the 0.360 deterministic safe floor) with 4 deaths; A0 0.296 with 35 deaths and A0s 0.222 with 39; the healthy-link controls give A0 0.275/13 deaths and A1 0.511/0. Per-seed service overlaps, so we draw no service significance. The live agent loop was not wired to the local guard, so the four A1 seed-1 deaths are not cleared by the deterministic RQ of §7.5.

Accurate call accounting. The 1089 decisions are not 1089 successful calls: 52 ended in a parse-failure hold from `max_tokens` cutting the JSON mid-string, and 412 records carry a first-attempt error even when the retry succeeded (a logging artefact). Per-arm parse-failure rates are A0 7.1% (28/396), A0s 6.4% (19/297), A1 1.3% (5/396), so arm differences mix output-format reliability with any semantic effect.

The interface fault, symmetrically scored. The observation’s `link` field carries only LoRa access receipts (`n_heard`) and no backhaul state. In the forced outage window, A0/A0s assert “link up / commands can apply” on 36/32 decisions (A1 on 1 after token-boundary de-noising of an “uplink-only” substring)—deterministically over-optimistic, since the primary is down throughout; these accompany the ineffective dense orders and deaths. In the opposite direction, the v4 certificate asserted “control plane not delivering” whenever fewer than all nodes were heard, firing at five healthy-link times when the primary was in fact usable, twice after 13/14 nodes had already confirmed dense. The earlier “false-report” tally was invalid on three counts: it only ran on decisions that ordered dense (vacuous for the mostly-hold A1), never ran on the healthy group (initialisation zeros), and counted keywords the certificate prompt itself taught (such terms appear in 317 A1 decisions and zero A0/A0s decisions). Offline replay of an unknown-aware v5 projector on the identical snapshots issues zero unreachable assertions without a network-side field (all five healthy false alarms become unconfirmed) and splits 528 blanket night kills into 189 genuinely energy-infeasible and 339 advisory (every near-dawn 04:30–06:00 case is advisory). The honest finding is that a missing backhaul-state signal causes bidirectional errors; isolating any independent increment of the projector requires a matched-capability arm that receives the same constants and heuristics without the projector, end-to-end v5 runs with an explicit backhaul-state field, and cross-model replication, all left as future work.

8 Related work

LLMs in emergency and network control. TopoLLM applies adaptive tool learning to post-disaster topology planning [11]; later systems let LLMs schedule UAV paths and bandwidth, shape RL rewards, or orchestrate recovery tools, including intent objects, budget/time constraints, candidate-plan checks, local repair, and documented network-management APIs [12, 13]. We therefore claim no novelty for “natural language to constraints” or tool orchestration. These decide network resources or compose recovery tools under an assumed reachable control path; they do not terminate state that must keep acting after that path dies, and do not maintain an unknown-aware three-class state over a long routine monitoring workflow. Agentic Wi-Fi likewise uses an LLM as a solver for a separately defined coordination protocol; we follow that ordering—problem first, agent optional.

Scheduling, freshness, energy. The closest state-model precedent is an LLM in-context data-collection scheduler with bounded per-sensor queues, exogenous arrivals, finite service opportunities, and feedback [14]; AoI and energy-harvesting control, including instantaneous-battery threshold policies and partially observed harvesting, address resource pieces [15, 16, 7]; overbooking and contact-aware forwarding for space links [17] and network-calculus bounds [18] are complementary deterministic characterisations. Joint sampling/transmission/energy optimisation is established and we do not claim it; our distinction is persistent *remote* configuration with Class A apply latency, a separated access vs. primary/backup return path, and termination under a control outage. Consistent network update research has long studied per-update validity, dependencies, and safe transitions [19]; we rely on that lesson (versioning and dependencies are not new) and add the disconnected, energy-constrained edge case in which an installed update’s *own data* become undeliverable before a counter-update can arrive.

Packet discard, DTN expiry, contacts. AoI management pre-empts or drops old updates [8, 20]; deadline-aware buffer discard has been coupled with adaptive ARQ for video [2]; DTN bundle protocols attach creation-time lifetimes and use TTL/encounter drop [9, 21, 3], and custody transfer [22] moves retention responsibility as a forward hand-off (since migrated out of

the BPv7 core). Rural LoRa-LEO systems schedule around estimated satellite contacts and exploit spatial gateway diversity [1]. Our record disposition *is* BPv7-style expiry and we prove the equivalence; our configuration lease is a TTL/lease primitive with a delivery-derived bound. The claimed deltas are narrow and explicit: an end-to-end business-deadline basis with cross-segment source/gateway consistency (expired backhaul suppression alone back-pressures access), and a configuration bound computed from the last deliverable slot so that termination is safe across outage phases without a live control path.

9 Discussion, limitations, and threats to validity

Scope. The throughput negative result and the resource walls are scoped to the policy families and one operating point; they are not an optimum, and a better scheduler could exist. The lease result is deterministic over two constructed phases and five harvest levels; it establishes the mechanism’s existence and the failure of fixed clock/TTL bounds across those phases, not a universal parameter choice. Wider sweeps of backup rate/payload, node count, capacity, cloud model, restart thresholds, and non-absorbing brownout are future work and could move the energy wall (not the slot/capacity walls). The lease bound uses information valid at install time; an authorisation changed entirely during a long outage is physically unreachable, and the correct behaviour is safe-default revert plus an unfulfillable declaration, which we report rather than hide. **Model simplifications.** Central acknowledgement is synchronous and airtime-free, equally favouring every queue rule; gateway acknowledgements would consume Class A windows in deployment. During Episode I the node tests deadlines against the public pre-change cadence, with post-upgrade deadlines used only in offline accounting. Single fixed-period missions make obligation-keyed and deadline-keyed record release coincide; multi-obligation or overlapping mission versions require release keyed to a record’s latest still-valid deadline. Episode II is a controlled scenario for the configuration sub-problem, not an independent field measurement. **Agent evidence.** The live study is single-model, three seeds per arm, temperature-0, with large per-run variance and a confounded expert packet; it demonstrates the interface fault and the projector’s offline effect, not an independent agent increment. The hardened harness (per-request accounting, symmetric scoring, unknown-aware projector and energy ledger, default-off local guard and lease) is in the repository so matched-capability, end-to-end v5, and cross-model studies can run without re-litigating instrumentation; those are explicitly not claimed here. **Provenance.** One standards channel was obtained from a document-sharing host (top-grade document, lower-grade copy chain) because the official host was unreachable by TLS; clause numbers were cross-checked from two independent copies, and unverified international LoRa-plus-satellite and vendor lines are not used as fact. We did not execute cited systems’ implementations and claim no replication of them.

10 Conclusion

After a backhaul failure, the communication state installed beforehand keeps executing, and its termination cannot wait for the control path that failed. We formulated this as one local termination problem over two kinds of persistent installed state. Records carry a locally certain release time—the business deadline—and standard per-record expiry, placed consistently at the source, doubles outage on-time delivery while gateway-only suppression merely moves cost to the access segment. Configurations carry a release time the node cannot observe during an outage; a fixed clock guard or fixed TTL either kills nodes under overcast skies or falsely releases deliverable obligations across phases, whereas a delivery-bounded lease—an ordinary lease whose bound is the last deliverable

slot, installed while connected and enforced locally—removes those deaths without false release. Throughout, mechanisms act only on evidence the acting entity lawfully holds, say unknown when they cannot tell, and declare work unfulfillable rather than complete. The network problem and its deterministic solutions stand on their own; an LLM is one candidate controller, and our live study shows most clearly the interface fault—access receipts mistaken for backhaul state—that the evidence discipline is designed to prevent.

References

- [1] Yidong Ren, Amalinda Gamage, Li Liu, Mo Li, Shigang Chen, Younsuk Dong, and Zhichao Cao. SateRIoT: High-performance ground-space networking for rural IoT. In *Proc. 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom '24)*, 2024.
- [2] Rouzbeh Razavi, Martin Fleury, and Mohammed Ghanbari. Deadline-aware video delivery in a disrupted Bluetooth network. In *Proc. IEEE Sarnoff Symposium*, 2007. IEEE Xplore document 4567352.
- [3] IETF Delay-Tolerant Networking Working Group. Bundle protocol version 7. Technical Report RFC 9171, IETF / RFC Editor, 2022.
- [4] Ministry of Natural Resources of the People’s Republic of China. DZ/T 0460–2023, technical specification for geological-disaster monitoring (clauses 5.3.3, 5.3.6, 5.3.7, 8.4.1, 8.4.2). Technical report, Industry standard of the People’s Republic of China, 2023.
- [5] Ministry of Natural Resources of the People’s Republic of China. DZ/T 0450–2023, communication-layer specification for geological-disaster monitoring (clauses 7.4.2.3, 7.4.3.2, 7.4.3.5). Technical report, Industry standard of the People’s Republic of China, 2023.
- [6] China Satellite Navigation Office. BDS-OS-PS-3.0, beidou navigation satellite system open-service short-message communication service specification (table 7-1). Technical report, China Satellite Navigation Office, 2023.
- [7] Maice Costa, Marian Codreanu, and Anthony Ephremides. On the age of the information in status update systems with packet management. *IEEE Transactions on Information Theory*, 62(4):1897–1910, 2016. Conference version in *Proc. IEEE ISIT 2014*, pp. 1583–1587.
- [8] Chih ping Kam, Sennur Kaya, and Anthony Ephremides. Controlling the age of information: Buffer size, deadline, and packet replacement. In *Proc. IEEE Military Communications Conference (MILCOM)*, 2016.
- [9] Kevin Fall, Scott Burleigh, Adrian Hooke, et al. Delay-tolerant networking architecture. Technical Report RFC 4838, IRTF / RFC Editor, 2007.
- [10] Takahiro Wada, Hayato Kitamoto, Takerou Fukuhara, and Iwao Sasase. A transport protocol with energy consumption reduction by reducing redundant transmissions in wireless sensor and actor networks. In *Proc. IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*, 2009. IEEE Xplore document 5291354.
- [11] Yizhuo Ma, Rongzheng Wang, Ke Qin, et al. TopoLLM: LLM-driven adaptive tool learning for real-time emergency network topology planning. *Digital Communications and Networks*, 12:273–282, 2026. Advance online publication, 1 October 2025.

- [12] Jinyin Bai, Wei Zhu, Xiangchen Wang, Shiluo Guo, Zongzhe Nie, Tianjin Ni, Jinji Zhou, Kaiyang Kou, Lingxin Xu, and Yihao Zhong. ICG-Restore: Intent-constrained, graph-enhanced LLM planning with minimal-edit repair for post-disaster emergency communication recovery. *AI*, 7(8):294, 2026.
- [13] Abdelkader Mekrache, Adlen Ksentini, and Christos Verikoukis. OSS-GPT: An LLM-powered intent-driven operations support system for 6G networks. In *Proc. IEEE Conference on Network Softwarization (NetSoft)*, 2025.
- [14] Yousef Emami, Hao Zhou, SeyedSina Nabavirazani, and Luis Almeida. LLM-enabled in-context learning for data collection scheduling in UAV-assisted sensor networks. *IEEE Internet of Things Journal*, 2025. arXiv:2504.14556.
- [15] Baran Tan Bacinoglu, Yin Sun, Elif Uysal, and Volkan Mutlu. Optimal status updating with a finite-battery energy harvesting source. *arXiv preprint arXiv:1905.06679*, 2019.
- [16] Abolfazl Zakeri, Mohammad Moltafet, and Marian Codreanu. Semantic-aware sampling and transmission in energy harvesting systems: A POMDP approach. *arXiv preprint arXiv:2311.06522*, 2023. Preliminary results at Asilomar 2023 and IEEE GLOBECOM 2023.
- [17] Consultative Committee for Space Data Systems. Schedule-aware bundle routing. Technical Report CCSDS 734.3-B-1, Blue Book, Issue 1, CCSDS, 2019.
- [18] Jean-Yves Le Boudec and Patrick Thiran. *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*. Springer, LNCS 2050, 2001.
- [19] Klaus-Tycho Förster, Ratul Mahajan, and Roger Wattenhofer. Consistent updates in software defined networks: On dependencies, loop freedom, and blackholes. In *Proc. IFIP Networking Conference and Workshops*, pages 1–9, Vienna, Austria, 2016.
- [20] Ashok Krishnan K. S. and Vinod Sharma. Minimizing age of information in a multihop wireless network. *arXiv preprint arXiv:1909.06571*, 2019.
- [21] Saeid Iranmanesh. A novel queue management policy for delay-tolerant networks. *EURASIP Journal on Wireless Communications and Networking*, 2016(88):1–23, 2016.
- [22] Kevin Scott and Scott Burleigh. Bundle protocol specification. Technical Report RFC 5050, IRTF / RFC Editor, 2007.